7

Speech Recognition

In this chapter, we will cover the following recipes:

- Reading and plotting audio data
- Transforming audio signals into the frequency domain
- Generating audio signals with custom parameters
- Synthesizing music
- Extracting frequency domain features
- Building Hidden Markov Models
- Building a speech recognizer

Introduction

Speech recognition refers to the process of recognizing and understanding spoken language. Input comes in the form of audio data, and the speech recognizers will process this data to extract meaningful information from it. This has a lot of practical uses, such as voice controlled devices, transcription of spoken language into words, security systems, and so on.

Speech signals are very versatile in nature. There are many variations of speech in the same language. There are different elements to speech, such as language, emotion, tone, noise, accent, and so on. It's difficult to rigidly define a set of rules that can constitute speech. Even with all these variations, humans are really good at understanding all of this with relative ease. Hence, we need machines to understand speech in the same way.

Over the last couple of decades, researchers have worked on various aspects of speech, such as identifying the speaker, understanding words, recognizing accents, translating speech, and so on. Among all these tasks, automatic speech recognition has been the focal point of attention for many researchers. In this chapter, we will learn how to build a **speech recognizer**.

2016. Packt Publishing.
All rights reserved. May not be reproduced in any form without permission from the publisher, except fair uses permitted under U.S. or applicable copyright law.

EBSCO Publishing: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD 1285067; Prateek Joshi, Vahid Mirjalili; Python Machine Learning Cookbook : 100 Recipes That Teach You How to Perform Various Machine Learning Tasks in the Real World Account:ehost.

Reading and plotting audio data

Let's take a look at how to read an audio file and visualize the signal. This will be a good starting point, and it will give us a good understanding about the basic structure of audio signals. Before we start, we need to understand that audio files are digitized versions of actual audio signals. Actual audio signals are complex continuous-valued waves. In order to save a digital version, we sample the signal and convert it into numbers. For example, speech is commonly sampled at 44100 Hz. This means that each second of the signal is broken down into 44100 parts, and the values at these timestamps are stored. In other words, you store a value every 1/44100 seconds. As the sampling rate is high, we feel that the signal is continuous when we listen to it on our media players.

How to do it...

1. Create a new Python file, and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
```

2. We will use the `wavfile` package to read the audio file from the `input_read.wav` input file that is already provided to you:

```
# Read the input file
sampling_freq, audio = wavfile.read('input_read.wav')
```

3. Let's print out the parameters of this signal:

```
# Print the params
print '\nShape:', audio.shape
print 'Datatype:', audio.dtype
print 'Duration:', round(audio.shape[0] / float(sampling_freq),
3), 'seconds'
```

4. The audio signal is stored as 16-bit signed integer data. We need to normalize these values:

```
# Normalize the values
audio = audio / (2.**15)
```

5. Let's extract the first 30 values to plot, as follows:

```
# Extract first 30 values for plotting
audio = audio[:30]
```

6. The X-axis is the time axis. Let's build this axis, considering the fact that it should be scaled using the sampling frequency factor:

```
# Build the time axis
x_values = np.arange(0, len(audio), 1) / float(sampling_freq)
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

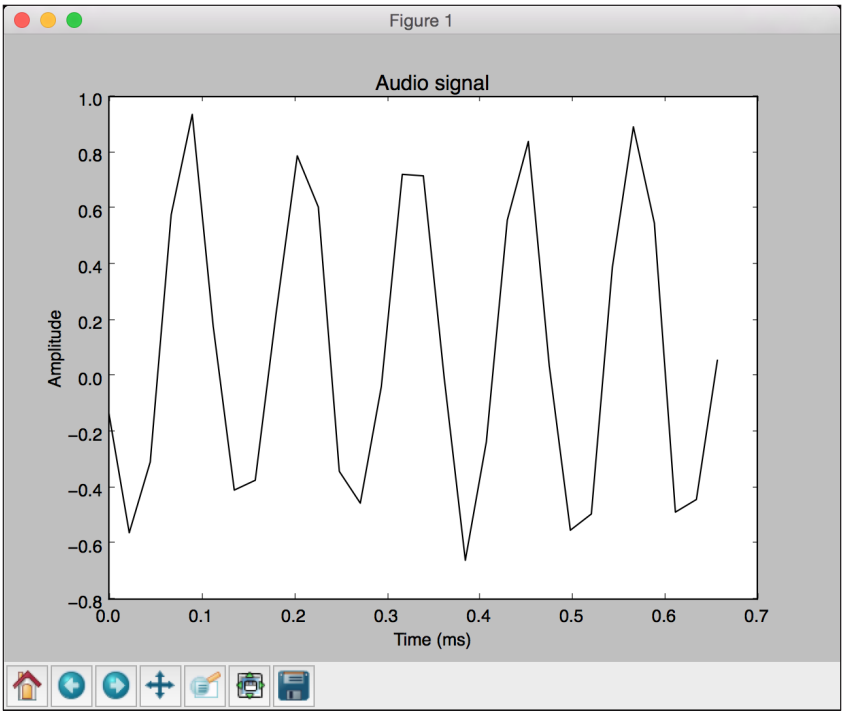
7. Convert the units to seconds:

```
# Convert to seconds
x_values *= 1000
```

8. Let's plot this as follows:

```
# Plotting the chopped audio signal
plt.plot(x_values, audio, color='black')
plt.xlabel('Time (ms)')
plt.ylabel('Amplitude')
plt.title('Audio signal')
plt.show()
```

9. The full code is in the `read_plot.py` file. If you run this code, you will see the following signal:



10. You will also see the following printed on your Terminal:

```
Shape: (132300,)
Datatype: int16
Duration: 3.0 seconds
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Transforming audio signals into the frequency domain

Audio signals consist of a complex mixture of sine waves of different frequencies, amplitudes, and phases. Sine waves are also referred to as **sinusoids**. There is a lot of information that is hidden in the frequency content of an audio signal. In fact, an audio signal is heavily characterized by its frequency content. The whole world of speech and music is based on this fact. Before you proceed further, you will need some knowledge about **Fourier transforms**. A quick refresher can be found at `http://www.thefouriertransform.com`. Now, let's take a look at how to transform an audio signal into the frequency domain.

How to do it...

1. Create a new Python file, and import the following package:

```
import numpy as np
from scipy.io import wavfile
import matplotlib.pyplot as plt
```

2. Read the `input_freq.wav` file that is already provided to you:

```
# Read the input file
sampling_freq, audio = wavfile.read('input_freq.wav')
```

3. Normalize the signal, as follows:

```
# Normalize the values
audio = audio / (2.**15)
```

4. The audio signal is just a NumPy array. So, you can extract the length using the following code:

```
# Extract length
len_audio = len(audio)
```

5. Let's apply the Fourier transform. The Fourier transform signal is mirrored along the center, so we just need to take the first half of the transformed signal. Our end goal is to extract the power signal. So, we square the values in the signal in preparation for this:

```
# Apply Fourier transform
transformed_signal = np.fft.fft(audio)
half_length = np.ceil((len_audio + 1) / 2.0)
transformed_signal = abs(transformed_signal[0:half_length])
transformed_signal /= float(len_audio)
transformed_signal **= 2
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

6. Extract the length of the signal:

```
# Extract length of transformed signal
len_ts = len(transformed_signal)
```

7. We need to double the signal according to the length of the signal:

```
# Take care of even/odd cases
if len_audio % 2:
    transformed_signal[1:len_ts] *= 2
else:
    transformed_signal[1:len_ts-1] *= 2
```

8. The power signal is extracted using the following formula:

```
# Extract power in dB
power = 10 * np.log10(transformed_signal)
```

9. The X-axis is the time axis. We need to scale this according the sampling frequency and then convert this into seconds:

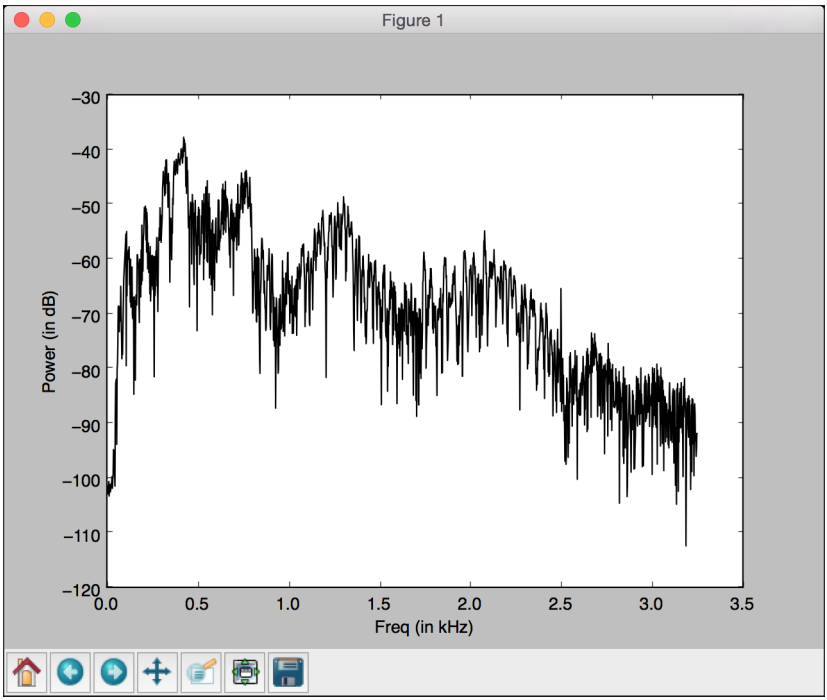
```
# Build the time axis
x_values = np.arange(0, half_length, 1) * (sampling_freq / len_
audio) / 1000.0
```

10. Plot the signal, as follows:

```
# Plot the figure
plt.figure()
plt.plot(x_values, power, color='black')
plt.xlabel('Freq (in kHz)')
plt.ylabel('Power (in dB)')
plt.show()
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

11. The full code is in the `freq_transform.py` file. If you run this code, you will see the following figure:



Generating audio signals with custom parameters

We can use NumPy to generate audio signals. As we discussed earlier, audio signals are complex mixtures of sinusoids. So, we will keep this in mind when we generate our own audio signal.

How to do it...

1. Create a new Python file, and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io.wavfile import write
```

2. We need to define the output file where the generated audio will be stored:

```
# File where the output will be saved
output_file = 'output_generated.wav'
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

3. Let's specify the audio generation parameters. We want to generate a three-second long signal with a sampling frequency of `44100` and a tonal frequency of `587` Hz. The values on the time axis will go from *-2*pi* to *2*pi*:

```
# Specify audio parameters
duration = 3  # seconds
sampling_freq = 44100  # Hz
tone_freq = 587
min_val = -2 * np.pi
max_val = 2 * np.pi
```

4. Let's generate the time axis and the audio signal. The audio signal is a simple sinusoid with the previously mentioned parameters:

```
# Generate audio
t = np.linspace(min_val, max_val, duration * sampling_freq)
audio = np.sin(2 * np.pi * tone_freq * t)
```

5. Let's add some noise to the signal:

```
# Add some noise
noise = 0.4 * np.random.rand(duration * sampling_freq)
audio += noise
```

6. We need to scale the values to 16-bit integers before we store them:

```
# Scale it to 16-bit integer values
scaling_factor = pow(2,15) - 1
audio_normalized = audio / np.max(np.abs(audio))
audio_scaled = np.int16(audio_normalized * scaling_factor)
```

7. Write this signal to the output file:

```
# Write to output file
write(output_file, sampling_freq, audio_scaled)
```

8. Plot the signal using the first 100 values:

```
# Extract first 100 values for plotting
audio = audio[:100]
```

9. Generate the time axis:

```
# Build the time axis
x_values = np.arange(0, len(audio), 1) / float(sampling_freq)
```

10. Convert the time axis into seconds:

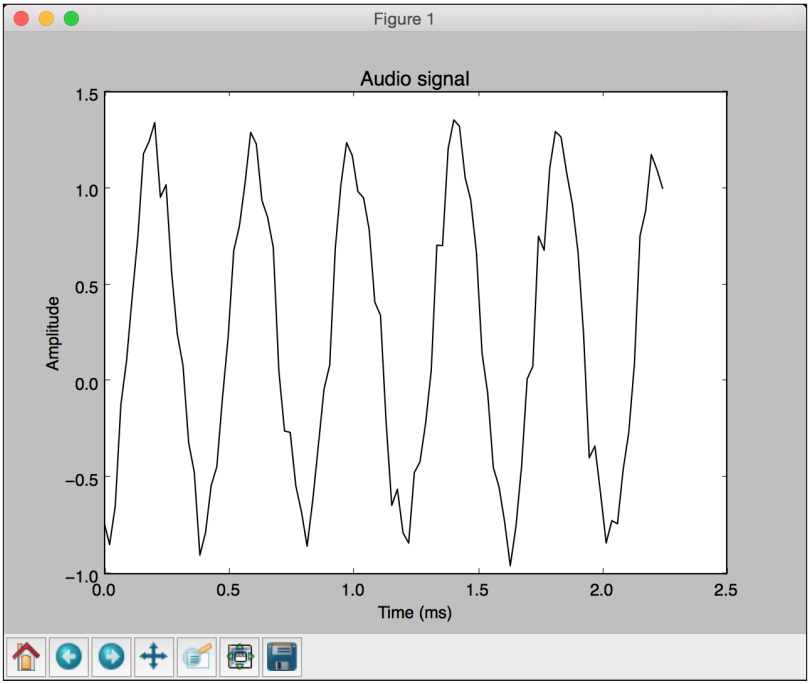
```
# Convert to seconds
x_values *= 1000
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

11. Plot the signal, as follows:

```
# Plotting the chopped audio signal
plt.plot(x_values, audio, color='black')
plt.xlabel('Time (ms)')
plt.ylabel('Amplitude')
plt.title('Audio signal')
plt.show()
```

12. The full code is in the `generate.py` file. If you run this code, you will get the following figure:



Synthesizing music

Now that we know how to generate audio, let's use this principle to synthesize some music. You can check out this link, `http://www.phy.mtu.edu/~suits/notefreqs.html`. This link lists various notes, such as *A*, *G*, *D*, and so on, along with their corresponding frequencies. We will use this to generate some simple music.

EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

How to do it...

1. Create a new Python file, and import the following packages:

```
import json
import numpy as np
from scipy.io.wavfile import write
import matplotlib.pyplot as plt
```

2. Define a function to synthesize a tone, based on input parameters:

```
# Synthesize tone
def synthesizer(freq, duration, amp=1.0, sampling_freq=44100):
```

3. Build the time axis values:

```
    # Build the time axis
    t = np.linspace(0, duration, duration * sampling_freq)
```

4. Construct the audio sample using the input arguments, such as amplitude and frequency:

```
    # Construct the audio signal
    audio = amp * np.sin(2 * np.pi * freq * t)

    return audio.astype(np.int16)
```

5. Let's define the `main` function. You have been provided with a JSON file called `tone_freq_map.json`, which contains some notes along with their frequencies:

```
if __name__=='__main__':
    tone_map_file = 'tone_freq_map.json'
```

6. Load that file:

```
    # Read the frequency map
    with open(tone_map_file, 'r') as f:
        tone_freq_map = json.loads(f.read())
```

7. Let's assume that we want to generate a G note for a duration of `2` seconds:

```
    # Set input parameters to generate 'G' tone
    input_tone = 'G'
    duration = 2     # seconds
    amplitude = 10000
    sampling_freq = 44100    # Hz
```

8. Call the function with the following parameters:

```
    # Generate the tone
    synthesized_tone = synthesizer(tone_freq_map[input_tone],
duration, amplitude, sampling_freq)
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

9. Write the generated signal into the output file:

```
# Write to the output file
write('output_tone.wav', sampling_freq, synthesized_tone)
```

10. Open this file in a media player and listen to it. That's the *G* note! Let's do something more interesting. Let's generate some notes in sequence to give it a musical feel. Define a note sequence along with their durations in seconds:

```
# Tone-duration sequence
tone_seq = [('D', 0.3), ('G', 0.6), ('C', 0.5), ('A', 0.3),
('Asharp', 0.7)]
```

11. Iterate through this list and call the synthesizer function for each of them:

```
# Construct the audio signal based on the chord sequence
output = np.array([])
for item in tone_seq:
    input_tone = item[0]
    duration = item[1]
    synthesized_tone = synthesizer(tone_freq_map[input_tone],
duration, amplitude, sampling_freq)
    output = np.append(output, synthesized_tone, axis=0)
```

12. Write the signal to the output file:

```
# Write to the output file
write('output_tone_seq.wav', sampling_freq, output)
```

13. The full code is in the `synthesize_music.py` file. You can open the `output_tone_seq.wav` file in your media player and listen to it. You can feel the music!

Extracting frequency domain features

We discussed earlier how to convert a signal into the frequency domain. In most modern speech recognition systems, people use frequency-domain features. After you convert a signal into the frequency domain, you need to convert it into a usable form. **Mel Frequency Cepstral Coefficients** (**MFCC**) is a good way to do this. MFCC takes the power spectrum of a signal and then uses a combination of filter banks and discrete cosine transform to extract features. If you need a quick refresher, you can check out `http://practicalcryptography.com/miscellaneous/machine-learning/guide-mel-frequency-cepstral-coefficients-mfccs`. Make sure that the `python_speech_features` package is installed before you start. You can find the installation instructions at `http://python-speech-features.readthedocs.org/en/latest`. Let's take a look at how to extract MFCC features.

EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

How to do it...

1. Create a new Python file, and import the following packages:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from features import mfcc, logfbank
```

2. Read the `input_freq.wav` input file that is already provided to you:

```
# Read input sound file
sampling_freq, audio = wavfile.read("input_freq.wav")
```

3. Extract the MFCC and filter bank features:

```
# Extract MFCC and Filter bank features
mfcc_features = mfcc(audio, sampling_freq)
filterbank_features = logfbank(audio, sampling_freq)
```

4. Print the parameters to see how many windows were generated:

```
# Print parameters
print '\nMFCC:\nNumber of windows =', mfcc_features.shape[0]
print 'Length of each feature =', mfcc_features.shape[1]
print '\nFilter bank:\nNumber of windows =', filterbank_features.
shape[0]
print 'Length of each feature =', filterbank_features.shape[1]
```

5. Let's visualize the MFCC features. We need to transform the matrix so that the time domain is horizontal:

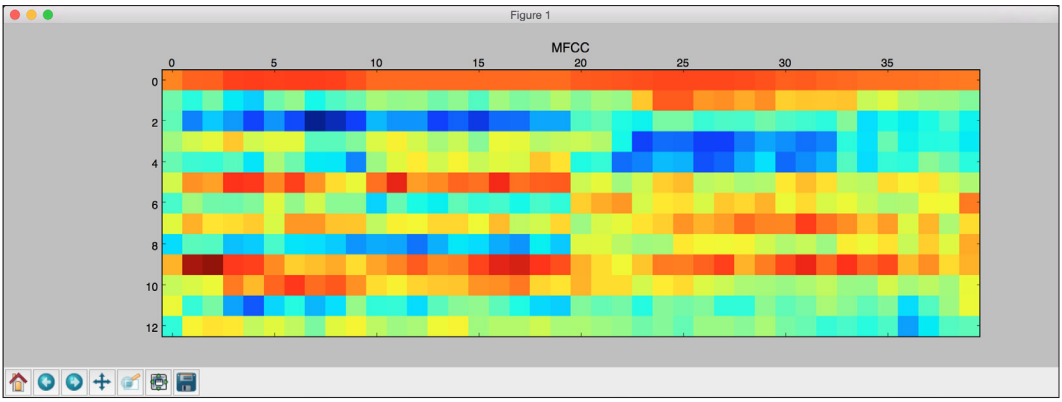
```
# Plot the features
mfcc_features = mfcc_features.T
plt.matshow(mfcc_features)
plt.title('MFCC')
```

6. Let's visualize the filter bank features. Again, we need to transform the matrix so that the time domain is horizontal:

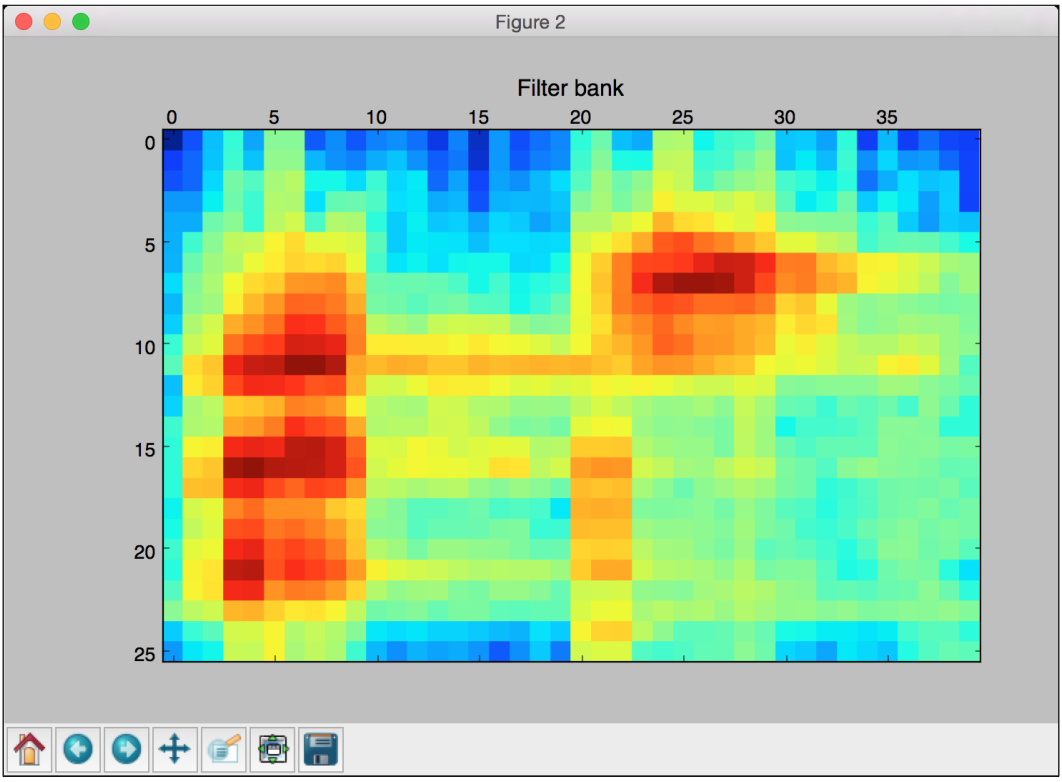
```
filterbank_features = filterbank_features.T
plt.matshow(filterbank_features)
plt.title('Filter bank')

plt.show()
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

7. The full code is in the `extract_freq_features.py` file. If you run this code, you will get the following figure for MFCC features:



8. The filter bank features will look like the following:



EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

9. You will get the following output on your Terminal:

```
MFCC:
Number of windows = 40
Length of each feature = 13

Filter bank:
Number of windows = 40
Length of each feature = 26
```

Building Hidden Markov Models

We are now ready to discuss speech recognition. We will use **Hidden Markov Models** (**HMMs**) to perform speech recognition. HMMs are great at modeling time series data. As an audio signal is a time series signal, HMMs perfectly suit our needs. An HMM is a model that represents probability distributions over sequences of observations. We assume that the outputs are generated by hidden states. So, our goal is to find these hidden states so that we can model the signal. You can learn more about it at `https://www.robots.ox.ac.uk/~vgg/rg/slides/hmm.pdf`. Before you proceed, you need to install the `hmmlearn` package. You can find the installation instructions at `http://hmmlearn.readthedocs.org/en/latest`. Let's take a look at how to build HMMs.

How to do it...

1. Create a new Python file. Let's define a class to model HMMs:

```
# Class to handle all HMM related processing
class HMMTrainer(object):
```

2. Let's initialize the class. We will use Gaussian HMMs to model our data. The `n_components` parameter defines the number of hidden states. The `cov_type` defines the type of covariance in our transition matrix, and `n_iter` indicates the number of iterations it will go through before it stops training:

```
    def __init__(self, model_name='GaussianHMM', n_components=4,
cov_type='diag', n_iter=1000):
```

   The choice of the preceding parameters depends on the problem at hand. You need to have an understanding of your data in order to select these parameters in a smart way.

EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

3. Initialize the variables:

```
        self.model_name = model_name
        self.n_components = n_components
        self.cov_type = cov_type
        self.n_iter = n_iter
        self.models = []
```

4. Define the model with the following parameters:

```
        if self.model_name == 'GaussianHMM':
            self.model = hmm.GaussianHMM(n_components=self.n_
components,
                    covariance_type=self.cov_type, n_iter=self.n_
iter)
        else:
            raise TypeError('Invalid model type')
```

5. The input data is a NumPy array, where each element is a feature vector consisting of *k*-dimensions:

```
    # X is a 2D numpy array where each row is 13D
    def train(self, X):
        np.seterr(all='ignore')
        self.models.append(self.model.fit(X))
```

6. Define a method to extract the score, based on the model:

```
    # Run the model on input data
    def get_score(self, input_data):
        return self.model.score(input_data)
```

7. We built a class to handle HMM training and prediction, but we need some data to see it in action. We will use it in the next recipe to build a speech recognizer. The full code is in the `speech_recognizer.py` file.

Building a speech recognizer

We need a database of speech files to build our speech recognizer. We will use the database available at `https://code.google.com/archive/p/hmm-speech-recognition/downloads`. This contains seven different words, where each word has 15 audio files associated with it. This is a small dataset, but this is sufficient to understand how to build a speech recognizer that can recognize seven different words. We need to build an HMM model for each class. When we want to identify the word in a new input file, we need to run all the models on this file and pick the one with the best score. We will use the HMM class that we built in the previous recipe.

EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

How to do it...

1. Create a new Python file, and import the following packages:

```
import os
import argparse

import numpy as np
from scipy.io import wavfile
from hmmlearn import hmm
from features import mfcc
```

2. Define a function to parse the input arguments in the command line:

```
# Function to parse input arguments
def build_arg_parser():
    parser = argparse.ArgumentParser(description='Trains the HMM
classifier')
    parser.add_argument("--input-folder", dest="input_folder",
required=True,
            help="Input folder containing the audio files in
subfolders")
    return parser
```

3. Define the `main` function, and parse the input arguments:

```
if __name__=='__main__':
    args = build_arg_parser().parse_args()
    input_folder = args.input_folder
```

4. Initiate the variable that will hold all the HMM models:

```
    hmm_models = []
```

5. Parse the input directory that contains all the database's audio files:

```
    # Parse the input directory
    for dirname in os.listdir(input_folder):
```

6. Extract the name of the subfolder:

```
        # Get the name of the subfolder
        subfolder = os.path.join(input_folder, dirname)

        if not os.path.isdir(subfolder):
            continue
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

7. The name of the subfolder is the label of this class. Extract it using the following:

```
# Extract the label
label = subfolder[subfolder.rfind('/') + 1:]
```

8. Initialize the variables for training:

```
# Initialize variables
X = np.array([])
y_words = []
```

9. Iterate through the list of audio files in each subfolder:

```
        # Iterate through the audio files (leaving 1 file for
testing in each class)
        for filename in [x for x in os.listdir(subfolder) if
x.endswith('.wav')][:-1]:
```

10. Read each audio file, as follows:

```
# Read the input file
filepath = os.path.join(subfolder, filename)
sampling_freq, audio = wavfile.read(filepath)
```

11. Extract the MFCC features:

```
# Extract MFCC features
mfcc_features = mfcc(audio, sampling_freq)
```

12. Keep appending this to the `X` variable:

```
# Append to the variable X
if len(X) == 0:
    X = mfcc_features
else:
    X = np.append(X, mfcc_features, axis=0)
```

13. Append the corresponding label too:

```
# Append the label
y_words.append(label)
```

14. Once you have extracted features from all the files in the current class, train and save the HMM model. As HMM is a generative model for unsupervised learning, we don't need labels to build HMM models for each class. We explicitly assume that separate HMM models will be built for each class:

```
# Train and save HMM model
hmm_trainer = HMMTrainer()
hmm_trainer.train(X)
hmm_models.append((hmm_trainer, label))
hmm_trainer = None
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

15. Get a list of test files that were not used for training:

```
# Test files
input_files = [
        'data/pineapple/pineapple15.wav',
        'data/orange/orange15.wav',
        'data/apple/apple15.wav',
        'data/kiwi/kiwi15.wav'
        ]
```

16. Parse the input files, as follows:

```
# Classify input data
for input_file in input_files:
```

17. Read in each audio file:

```
    # Read input file
    sampling_freq, audio = wavfile.read(input_file)
```

18. Extract the MFCC features:

```
    # Extract MFCC features
    mfcc_features = mfcc(audio, sampling_freq)
```

19. Define variables to store the maximum score and the output label:

```
    # Define variables
    max_score = None
    output_label = None
```

20. Iterate through all the models and run the input file through each of them:

```
    # Iterate through all HMM models and pick
    # the one with the highest score
    for item in hmm_models:
        hmm_model, label = item
```

21. Extract the score and store the maximum score:

```
        score = hmm_model.get_score(mfcc_features)
        if score > max_score:
            max_score = score
            output_label = label
```

22. Print the true and predicted labels:

```
    # Print the output
    print "\nTrue:", input_file[input_file.find('/')+1:input_
file.rfind('/')]
    print "Predicted:", output_label
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

23. The full code is in the `speech_recognizer.py` file. If you run this code, you will see the following on your Terminal:

```
True: pineapple
Predicted: pineapple

True: orange
Predicted: orange

True: apple
Predicted: apple

True: kiwi
Predicted: kiwi
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 4/16/2026 6:13:43 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.